

PLAN DE DESPLIEGUE DE SOFTWARE CON ALTA DISPONIBILIDAD EMPRESA DE SERVICIOS PÚBLICOS

Universidad Tecnológica Nacional - Facultad Regional Córdoba

Cátedra Ingeniería y Calidad de Software - 4K3

Grupo 6: Cuestas, M.; Gallino, G.; Marchetti, M.; Marín, E.; Ocampo, G.; Padilla, T.; Picatto, R.; Pluda, A.; Veron, J.; Zin, C.



INTRODUCCIÓN

El despliegue de software es una fase crítica del ciclo de vida de sistemas en producción, especialmente en sectores sensibles como servicios públicos (electricidad, gas y agua). En estos entornos, es fundamental asegurar la continuidad operativa, disponibilidad 24/7, mecanismos de contingencia y robustez ante fallos. Este trabajo tiene como objetivo diseñar un plan de despliegue integral para un producto de software que gestiona la facturación, pagos, notificación y atención al cliente de una empresa de servicios públicos. El propósito es garantizar que el sistema esté disponible en todo momento, pueda desplegarse en la nube con redundancia local y minimice riesgos asociados a fallos.

HERRAMIENTAS



MÉTODO

Infraestructura y Redundancia (Nube Híbrida)

Modelo: Híbrido Activo-Pasivo.

- Sitio principal de cloud pública (nginx, microservicios, bases de datos replicadas).
- Sitio de respaldo local de la empresa (Backup de base de datos cloud).
- Kubernetes (K8s).
- VPN Site-to-Site configurada entre el cloud y el servidor local para la replicación segura de la base de datos.

Artefactos y Prácticas CI/CD

Artefactos: Contenedores Docker inmutables, versionados con etiquetas semánticas.

Integración Continua (CI): Pruebas unitarias, de integración y análisis de código estático automatizados en cada commit.

Entrega Continua (CD): Pipeline orquestado por Jenkins que construye el artefacto, lo somete a pruebas de estrés, escaneo de vulnerabilidades y despliega automáticamente al entorno de Pre-Producción, si se aprueba por el Product Owner este pasa a Producción.

Estrategia de Despliegue

Rolling Update: Despliegue por defecto para actualizaciones menores. K8s reemplaza gradualmente las versiones antiguas de los Pods por los nuevos.

Blue/Green: Aplicado a los componentes más críticos. La nueva versión (Green) se despliega en paralelo a la versión actual (Blue) y el tráfico se conmuta solo cuando las pruebas post-despliegue son 100% exitosas.

RESULTADOS

Alta Disponibilidad y Autocuración

- Arquitectura de servicio:** Mínimo 3 réplicas por servicio crítico (Pods).
- Balanceo de carga:** Servicio de Kubernetes (Service) con Load Balancer integrado y nginx para balanceado interno.
- Base de datos:** Replicación Asíncrona cloud (Maestra) a Local (Esclava) y demás cloud (Esclava).

Ejecución del despliegue (Act. Tarifaria)

- Artefacto:** Imagen de Docker.
- Tipo de despliegue:** Blue/Green y Rolling Update según criticidad.
- Horario de mantenimiento:** Madrugada de Domingo a Lunes (03:00 - 05:00).
- Pruebas Post-Despliegue:** Pruebas de humo automatizadas y control de errores.
- Pre-requisitos de mantenimiento:** Backup de BDD completo y notificaciones previas de mantenimiento

Plan de Contingencia y Rollback

- Rollback automático:** Si las pruebas post-despliegue fallan o si Prometheus/Grafana detectan métricas críticas, K8s hará rollback automáticamente.
- Plan de contingencia:**
 - Promoción del servidor local.
 - Redirección de tráfico mediante Host Bastión.
- Comunicación:** Product Owner notifica a Soporte antes y después de un despliegue o rollback.

Respuesta Automática a Fallos (Autocuración y Rollback)

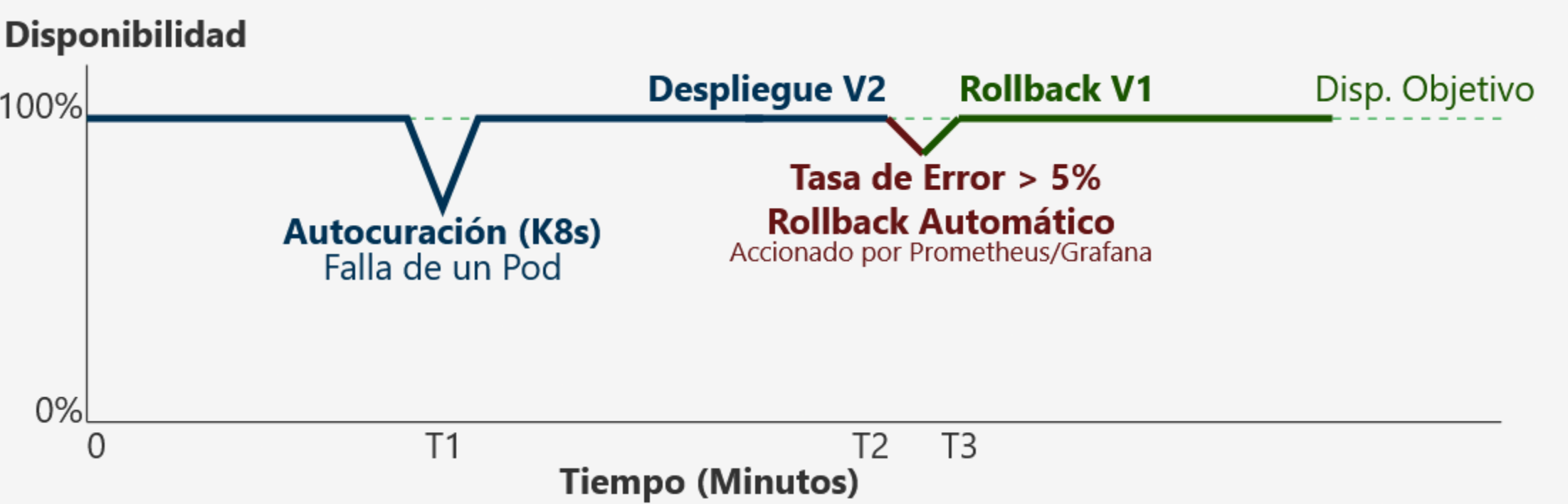
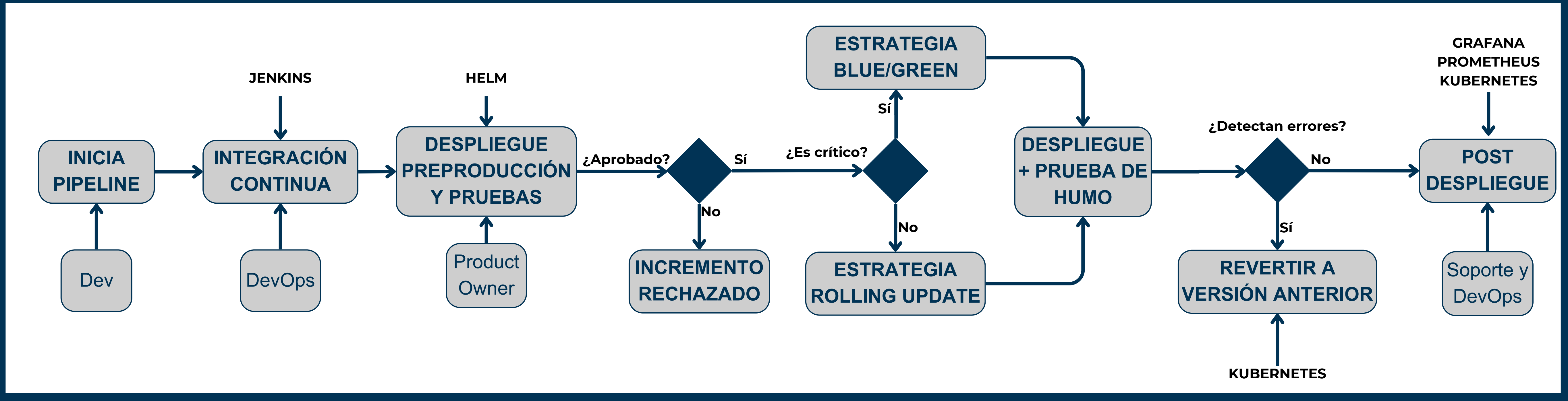


DIAGRAMA DE FLUJO



REFS.



CONCLUSIÓN

El despliegue de software en entornos críticos como los servicios públicos exige estrategias específicas que minimicen riesgos y aseguren disponibilidad continua. El plan desarrollado considera todas las etapas del proceso, desde la infraestructura hasta la validación post-release, integrando herramientas modernas de DevOps y prácticas de despliegue seguro. Este enfoque permite garantizar la confiabilidad del servicio, la trazabilidad de versiones y la capacidad de respuesta ante fallos. Como resultado, el sistema puede mantenerse activo sin interrupciones significativas, ofreciendo una experiencia de usuario robusta y confiable.